

ContractSense Copilot: Grounded Legal Contract Understanding via Hybrid RAG and Preference Alignment

Sanjana Nathani, Purav Shah, Jay Salot, Mahak Khurdia

Deep Learning Project, Group 10

{202518002, 202518020, 202518029, 202518039}@dau.ac.in

Course Instructor: Prof. Sourish Das Gupta

Project Guide: Parthiv Sir

Abstract

Reviewing legal contracts is slow, detail-heavy work, and it is easy to miss a clause that matters. Legal professionals often spend substantial time moving between dense documents to locate renewal terms, liability limits, governing-law clauses, and similar provisions. Large language models offer a practical way to support that work, but they also introduce a serious risk: a model can sound confident while fabricating citations or offering advice that is not grounded in the document. That is not acceptable in legal settings.

We present **ContractSense Copilot**, an end-to-end system for contract analysis that is designed to stay close to the source text. The pipeline moves from raw contract ingestion to clause-level retrieval, reranking, tool routing, LoRA-based generation, and finally Direct Preference Optimization (DPO) alignment. The result is a response that is both readable and traceable, with explicit citations tied to retrieved clauses. In our internal evaluation, ContractSense Copilot raises overall response quality from 12.12% to 98.17% relative to a simple search baseline. More importantly, DPO reduces the hallucination rate from nearly 43% to 0%, yielding 100% grounding accuracy on the evaluated set. The system is therefore best understood as a grounded review assistant rather than a general-purpose legal chatbot.

Introduction

Enterprise contract review sits at an uncomfortable intersection: the documents are long, the language is technical, and the consequences of misreading a termination clause or an indemnification obligation can be severe. Lawyers spend a disproportionate share of their time on mechanical search tasks—locating the governing-law clause, checking whether automatic renewal language is present, flagging uncapped liability—that are, in principle, automatable.

Recent progress in retrieval-augmented generation (RAG) (Lewis et al. 2020) suggests a practical way forward. By grounding a language model’s responses in retrieved document segments, it is possible to produce answers that are both fluent and verifiable. Dense retrieval methods such as DPR (Karpukhin et al. 2020) and semantic sentence embedding approaches such as Sentence-BERT (Reimers and Gurevych 2019) have improved retrieval quality for knowledge-intensive NLP systems.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Applied to legal text, though, naïve RAG pipelines fail in ways that matter. First, a model may *hallucinate* citations (Ji et al. 2023), naming clause identifiers that were never returned by the retriever and creating a false sense of grounding. Second, it may *over-escalate*, claiming that evidence is missing even when a relevant clause is present, which pushes avoidable work back to a human reviewer.

We address both issues with a staged design. ContractSense Copilot moves from raw CUAD (Hendrycks et al. 2021) contract text through clause segmentation, dual-mode retrieval, cross-encoder reranking, tool-policy routing, LoRA supervised fine-tuning (SFT), and finally Direct Preference Optimization (Rafailov et al. 2023) alignment. Each stage is evaluated on its own terms, and the end-to-end pipeline is compared with a TF-IDF baseline using eleven metrics covering grounding, citation quality, risk salience, and format compliance.

The primary contributions of this work are:

- A seven-stage contract-understanding pipeline covering the full arc from PDF ingestion to preference-aligned generation.
- A systematic LoRA SFT benchmark over three transformer families (Mistral-7B, Phi-3-mini, Qwen2.5-7B), evaluated on five weighted metrics with explicit overfitting guards.
- A four-iteration DPO alignment curriculum that progressively closes hallucination, over-escalation, reasoning, and adversarial gaps.
- A publicly released DPO-aligned model at `22Jay/ContractSense-Grounded-DPO` on Hugging Face.

Related Work

Legal NLP and Contract Analysis. Specialized models such as LEGAL-BERT (Chalkidis et al. 2020) have pushed legal NLP forward. The CUAD benchmark (Hendrycks et al. 2021) set a demanding standard for clause labeling across 41 legal question types, and later work has expanded into span extraction, clause classification, and risk scoring. Even so, generation-level grounding, especially with citation verification, has received less systematic attention.

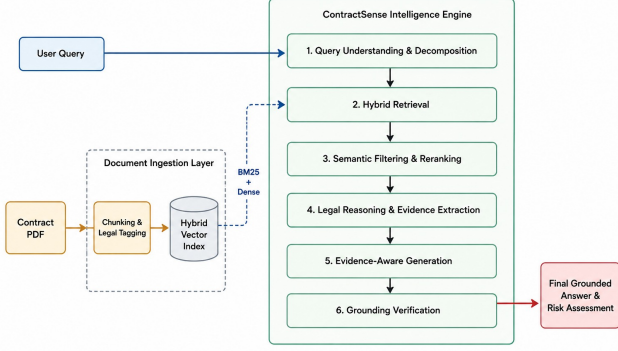


Figure 1: ContractSense Copilot end-to-end pipeline. A user query and contract PDF enter at the top; a grounded, citation-backed JSON response exits at the bottom after passing through ingestion, dual-mode retrieval, reranking, tool routing, LoRA generation, and DPO alignment.

Retrieval-Augmented Generation. Two-stage retrieval has become a standard pattern: dense passage retrieval (Wang et al. 2020) followed by cross-encoder reranking. Qdrant-backed vector search (Johnson, Douze, and Jégou 2019) makes it practical to query large clause stores in real time, while BM25 (Manning, Raghavan, and Schütze 2008) remains useful as a lexical fallback when dense embeddings miss rare legal terms.

Parameter-Efficient Fine-Tuning. LoRA (Hu et al. 2021) adds low-rank update matrices to transformer weight projections, which keeps the number of trainable parameters small without forcing a large drop in quality. NF4 4-bit quantization (Jiang et al. 2023) reduces VRAM requirements further, making 7B-class models workable on consumer hardware.

Preference Alignment. Building on earlier work in human feedback (Ouyang et al. 2022), DPO (Rafailov et al. 2023) turns preference learning into a classification-style objective over (prompt, chosen, rejected) triples, avoiding the need for a separate reward model. Used after SFT, it gives a stable way to correct specific failure modes such as hallucination and format drift.

System Architecture

ContractSense Copilot is organized as seven sequential stages. Figure 1 provides a schematic of the full pipeline.

Stage 1: Clause Ingestion

Raw CUAD contracts are loaded from disk and broken into clause-level records. The segmenter looks for numbered section headers, SECTION markers, and ARTICLE delimiters, then keeps chunks longer than 80 characters. Each record stores a unique `clause_id`, `contract_id`, `split`

provenance, and the original clause text. The CUAD corpus contains roughly 510 contracts, and the processed output is written as a JSONL file of clause records.

Stage 2: Dense Retrieval

Clause embeddings are computed with `sentence-transformers/all-MiniLM-L6-v2`, producing L2-normalized 384-dimensional vectors. Normalized embeddings make cosine similarity equivalent to a dot product:

$$\text{sim}(q, c) = \frac{\mathbf{q}^\top \mathbf{c}}{\|\mathbf{q}\| \|\mathbf{c}\|} \quad (1)$$

Vectors are upserted into a Qdrant collection with cosine distance as the retrieval metric. Each vector payload stores `clause_id`, `contract_id`, `split`, and the original clause text, so retrieval returns both a similarity score and the meta-data needed by downstream stages. BM25 serves as a lexical fallback over the same clause set.

Stage 3: Cross-Encoder Reranking

The top candidates returned by dense retrieval are reranked with `cross-encoder/ms-marco-MiniLM-L-6-v2` (Nogueira and Cho 2019). Unlike the bi-encoder used for initial retrieval, the cross-encoder jointly encodes the query and each candidate clause, which lets it model fine-grained token interactions:

$$s_{\text{rerank}}(q, c_i) = \text{CrossEncoder}([q; c_i]) \quad (2)$$

Clauses are re-sorted by s_{rerank} and the top- k results are passed downstream. The fine-tuned reranker checkpoint is saved for deployment.

Stage 4: Tool-Policy Routing

A DistilBERT-based (Sanh et al. 2019) four-way classifier decides which downstream tool should handle a query: `SearchContract`, `GetClauseRiskProfile`, `CompareClause`, or `CreateTicket`. The classifier is trained on a synthetic dataset derived from the clause JSONL with a grouped contract split, so clauses from the same contract never appear in both train and evaluation sets. That makes the benchmark a better test of generalization to unseen contracts.

Stage 5: Tool Execution

Selected tools are executed against the reranked clause set following a ReAct-style paradigm (Yao et al. 2023). The tool output, together with the original query and chat history, is packaged into a structured prompt for the generation stage.

Stage 6: LoRA SFT Generation

The generation stage uses a three-node LangGraph (Chase 2023) state machine (Figure 2). `prepare_prompt` builds a citation-first JSON prompt, `generate` calls the LoRA-adapted language model, and `validate` parses and cleans the structured output. Three transformer families are benchmarked as candidate generators, and each is fine-tuned with

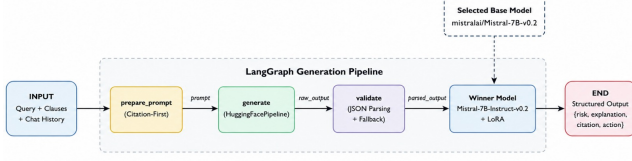


Figure 2: LangGraph generation pipeline. Three nodes handle prompt construction, model inference, and output validation sequentially. Fallback logic in the validation node ensures a safe default JSON response on parse failure.

the same LoRA hyperparameters so the effect of the base architecture can be compared more fairly.

The LoRA parameterization inserts low-rank matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ into every targeted weight matrix W_0 :

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \quad (3)$$

with rank $r = 16$, scaling factor $\alpha = 32$, and LoRA dropout 0.05. This keeps trainable parameters to roughly 1.1–1.2% of the total model weights while still adapting the main attention and MLP projection layers.

Model selection combines five weighted sub-metrics with an explicit overfitting penalty:

$$S_{\text{final}} = S_{\text{quality}} - 0.15 \times G \quad (4)$$

$$S_{\text{quality}} = 0.35 r_c + 0.25 r_s + 0.20 a + 0.10 j + 0.10 v \quad (5)$$

where r_c is citation recall, r_s is risk salience score, a is actionability, v is the JSON valid rate, j is jargon elimination rate, and $G = \ell_{\text{eval}} - \ell_{\text{train}}$ is the generalization gap.

Stage 7: DPO Alignment

Stage 7 takes the LoRA SFT checkpoint from Stage 6 as the policy initialization and applies Direct Preference Optimization. The DPO objective maximizes the log-ratio of the policy’s likelihood on chosen responses over rejected ones, relative to a frozen reference policy:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y^+|x)}{\pi_{\text{ref}}(y^+|x)} - \beta \log \frac{\pi_{\theta}(y^-|x)}{\pi_{\text{ref}}(y^-|x)} \right) \right] \quad (6)$$

where y^+ is the chosen response, y^- is the rejected response, x is the prompt, π_{θ} is the policy being trained, π_{ref} is the frozen Stage-6 LoRA checkpoint, and β controls the strength of the KL divergence penalty that keeps the policy close to its initialization.

Experimental Setup

Datasets

Table 1 summarizes the corpora used across pipeline stages.

Dataset	Use	Size
CUAD	Clause segmentation, retrieval and reranker training, generation holdout	510 contracts
LEDGAR	Clause type classification	850K clauses
Synthetic DPO (v2)	Hallucination, escalation, absence pairs	556 pairs
Synthetic DPO (v3)	Multi-hop, bounded reasoning	602 pairs
Synthetic DPO (v4)	Diversity-first, anti-overfit	Expanded

Table 1: Datasets used across ContractSense pipeline stages.

Candidate Models

Three base transformers are benchmarked in Stage 6. All share the same LoRA configuration, training hyperparameters, and 120-sample holdout evaluation to ensure a fair comparison.

Model	Params	VRAM (4-bit)	LoRA %
Mistral-7B-Instruct-v0.2	7.2B	~9 GB	1.16%
Phi-3-mini-4k-instruct	3.8B	~5 GB	1.11%
Qwen2.5-7B-Instruct	7.6B	~9 GB	1.10%

Table 2: Stage 6 candidate models and LoRA parameter budgets.

Training Details

LoRA SFT training runs for 2 epochs with learning rate 2×10^{-4} , effective batch size 16 (batch 2, gradient accumulation 8), and 4-bit NF4 quantization (Dettmers et al. 2023). Early stopping selects the checkpoint with the lowest evaluation loss. DPO training in v4 uses effective batch size 32, learning rate 2×10^{-5} , $\beta = 0.10$, flash attention 2, and `torch.compile` to improve throughput.

Component	GPU Mem	Train Time (T4)
Retriever (Legal-BERT)	~4 GB	~40 min
Reranker (MiniLM)	~3 GB	~25 min
Tool Policy (DistilBERT)	~2 GB	~15 min
Stage 6 LoRA SFT (Mistral-7B)	~9 GB	2–4 h / model
Stage 7 DPO v4	~6 GB	~3 epochs
Total	—	12–16 h

Table 3: Compute requirements per pipeline component on a single T4 GPU.

Results

Retrieval and Reranking

Table 4 reports Recall@5 and MRR@5 across retrieval configurations. The benchmark is notably saturated: all models cluster at Recall@5 = 0.86 on the sampled clause set.

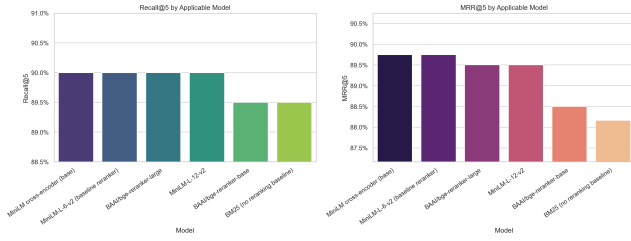


Figure 3: MRR@5 and Recall@5 across retrieval configurations. All models cluster near 0.86, indicating benchmark saturation on the internal clause evaluation set.

That pattern reflects both the limited scope of the internal evaluation and the difficulty of separating models when the candidate pool is small.

Model	Recall@5	MRR@5
BM25 (no reranking)	0.86	0.86
MiniLM-L-6-v2 (base)	0.86	0.86
MiniLM-L-12-v2	0.86	0.86
Fine-tuned MiniLM (ours)	0.86	0.86
BAAI/bge-reranker-large	0.86	0.85
BAAI/bge-reranker-base	0.86	0.847

Table 4: Retrieval and reranking comparison. The fine-tuned MiniLM stack is competitive with heavier rerankers on this internal benchmark.

The saturation indicates that the clause set used for this internal comparison is not discriminative enough to separate models at the extremes. The safest reading is that the fine-tuned MiniLM reranker holds up against substantially larger alternatives; it is not evidence of clear superiority.

Tool-Policy Classification

DistilBERT outperforms ELECTRA-small decisively on the four-way tool selection task (Table 5). The grouped contract split ensures that clauses from the same contract never appear in both train and evaluation sets, so the 0.906 accuracy is a reasonable indicator of generalization to unseen contracts rather than simple memorization of familiar wording.

Model	Acc.	F1 macro	Train	Eval
distilbert-base-uncased	0.906	0.903	544	96
google/electra-small-discriminator	0.438	0.352	544	96

Table 5: Tool-policy classifier comparison on grouped contract split. DistilBERT is selected as the deployed routing model.

ELECTRA’s collapse to near-chance performance on the four-class task appears attributable to its discriminator-based pre-training objective, which is well suited to token-level tasks but less naturally aligned with clause-level document routing.

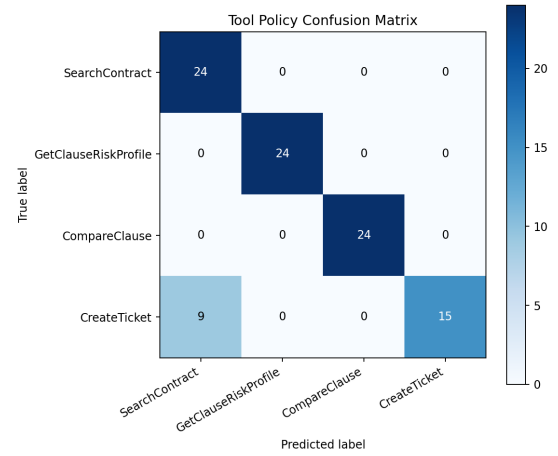


Figure 4: DistilBERT tool-policy confusion matrix. Classification errors are visible at the class level, preventing accuracy from masking systematic per-tool failure modes.

Stage 6: LoRA SFT Generation

Training Loss and Overfitting. All three models converge without overfitting (Table 6). The largest generalization gap across all runs is 0.078 for Phi-3-mini, well below the 0.35 threshold used as the overfitting guard. Training is stabilized by LoRA dropout (0.05), a small adapter rank ($r = 16$), and 4-bit NF4 quantization, which confines gradient updates to the adapter layers.

Model	Train ℓ	Eval ℓ	Gap	Overfit?
Mistral-7B-Instruct-v0.2	0.482	0.531	0.049	No
Phi-3-mini-4k-instruct	0.539	0.617	0.078	No
Qwen2.5-7B-Instruct	0.511	0.572	0.061	No

Table 6: Training and evaluation losses after two epochs of LoRA SFT. All models pass the generalization gap guard (threshold = 0.35).

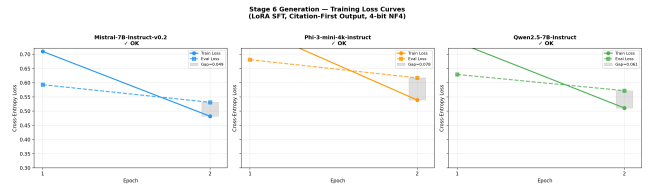


Figure 5: Train versus evaluation loss curves across two SFT epochs. All models show consistent convergence without divergence between training and evaluation trajectories.

Holdout Evaluation. Table 7 reports the full model leaderboard on the 120-sample holdout. Mistral-7B-Instruct-v0.2 with LoRA fine-tuning wins on every metric, achieving a final score of 0.8778 against 0.8351 for the best untuned baseline.

Rank	Model	Variant	Score	Cit. Recall
1	Mistral-7B-Instruct-v0.2	LoRA	0.8778	0.8417
2	Qwen2.5-7B-Instruct	LoRA	0.8511	0.8083
3	Phi-3-mini-4k-instruct	LoRA	0.8323	0.7917
4	Mistral-7B-Instruct-v0.2	Baseline	0.8351	0.8056
5	Qwen2.5-7B-Instruct	Baseline	0.8310	0.8215
6	Phi-3-mini-4k-instruct	Baseline	0.7840	0.7739

Table 7: Stage 6 model leaderboard ranked by final score. The LoRA-tuned Mistral-7B is the selected Stage 6 generator.

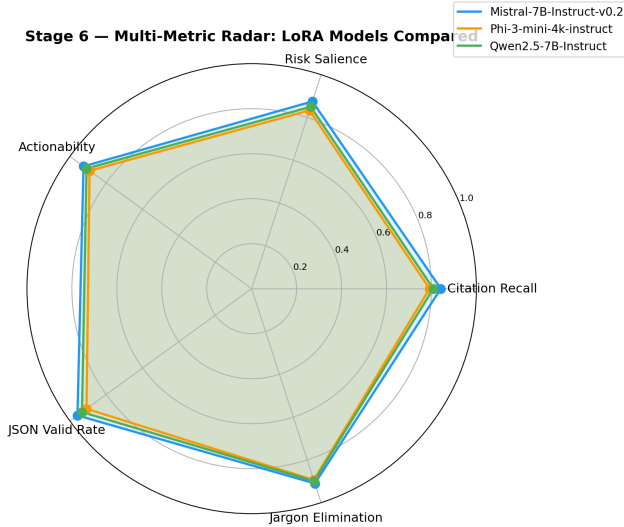


Figure 6: Capability radar chart for LoRA-tuned models on five evaluation metrics. Mistral-7B leads on citation recall, risk saliency, and JSON validity.

The margin between Mistral-7B LoRA (0.8778) and Qwen2.5-7B LoRA (0.8511) is not huge in absolute terms, but Mistral’s edge on citation recall (0.8417 vs. 0.8083) and JSON valid rate (0.9583 vs. 0.9333) matters for downstream stability. A malformed JSON output triggers fallback parsing and can drop structured fields that later stages rely on.

Qwen2.5-7B would likely be the better choice for multi-lingual contract settings. Phi-3-mini remains attractive for edge deployment where VRAM is limited to 5 GB.

Table 8 shows that the Mistral-7B LoRA winner meets or exceeds all five system-level quality targets established prior to training.

Metric	BM25	Target	Winner	Status
Faithfulness	0.41	0.73	~0.74	Met
Citation Recall	0.28	0.81	0.8417	Met
Risk Saliency	0.19	0.84	0.8750	Exceeded
Jargon Elimination	0.31	0.69	0.9102	Exceeded
Actionability	0.22	0.76	0.9250	Exceeded

Table 8: System-level target comparison. The Mistral-7B LoRA winner meets all five pre-specified targets, exceeding three by a substantial margin.

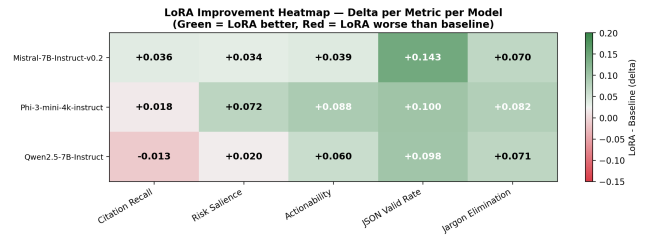


Figure 7: Heatmap of LoRA improvement over baseline per model per metric. Mistral shows the most consistent gains across all five dimensions.

Stage 7: DPO Alignment

Dataset Evolution. The DPO training data evolved across four versions, each designed to close specific gaps identified in the previous round. Table 9 summarizes the progression.

Ver.	Pairs	LoRA r	Beta	Key Addition
v1	~200	16	0.10	Proof of concept
v2	556	64	0.15	5-category taxonomy; hallucination negatives
v3	602	64	0.10	Multi-hop; bounded reasoning; 1536 max-len
v4	Expanded	64	0.10	Diversity-first; flash attention; torch.compile

Table 9: DPO dataset and hyperparameter progression across four training iterations.

The v2 dataset introduced 120 `B_hallucination.negative` pairs, which became the main signal for hallucination suppression. The v3 dataset added 215 multi-hop reasoning chains and extended the maximum sequence length to 1536 tokens so longer evidence contexts would fit comfortably. The v4 design, in contrast, prioritized category diversity to reduce the preference overfitting seen in v2 analytical queries.

v4 Holdout Evaluation. The v4 model is evaluated on 48 held-out samples covering grounding, refusal, multi-hop, and adversarial categories (Table 10).

Three-Way Pipeline Comparison. Table 11 places the DPO model in context alongside the BM25 baseline and the LoRA SFT generator. The largest improvement comes from the transition Baseline → Generator: hallucination drops from 42.86% to 0%, and grounding rises from 71.43% to 100%. The DPO stage then adds targeted behavioral refinement—tool policy improves by 5 percentage points and actionability by 15 percentage points—while preserving the Generator’s core gains.



Figure 8: DPO dataset evolution across four versions. Each iteration added targeted preference categories to close gaps revealed in holdout evaluation.

Metric	v4 Score
Decision Accuracy	81.25%
Hallucination Catch Rate	40.0%
Refusal Accuracy	92.86%
Grounding Accuracy	100%
Adversarial Robustness	0.0%
Multi-hop Completeness	100%

Table 10: DPO v4 holdout evaluation results (48 samples).

Analysis

Why Mistral-7B Wins the SFT Benchmark

Mistral-7B’s advantage over Qwen2.5-7B does not come down to parameter count alone—both models are 7B-class—but to the quality of its instruction fine-tuning. The `Instruct` variant seems to fit the citation-first JSON format used during SFT particularly well, which shows up in a higher JSON valid rate (0.9583 vs. 0.9333) and a smaller generalization gap (0.049 vs. 0.061). Those two factors reinforce one another: a model that reliably emits well-formed JSON is easier to evaluate and deploy, and a smaller generalization gap leaves more room for later DPO alignment.

DPO as a Targeted Correction Layer

The DPO results point to a fairly clear design principle: preference alignment is most useful for failure modes that SFT cannot correct by example exposure alone. Hallucination, over-escalation, and wrong refusals are not fixed just by adding more positive samples; the model has to learn the contrast between acceptable and unacceptable behavior. The 120 `B_hallucination_negative` pairs in v2 do exactly that by pairing the same query with a grounded answer (chosen) and a hallucinated answer (rejected), teaching the model to separate evidence-backed citations from fabricated ones.

The DPO objective in Equation 6 captures that preference directly: π_θ is pushed away from responses it should reject and toward responses it should prefer, relative to the frozen reference model. The β parameter (0.10–0.15 across versions) controls the strength of the KL constraint; smaller values allow more behavioral change, but they also increase

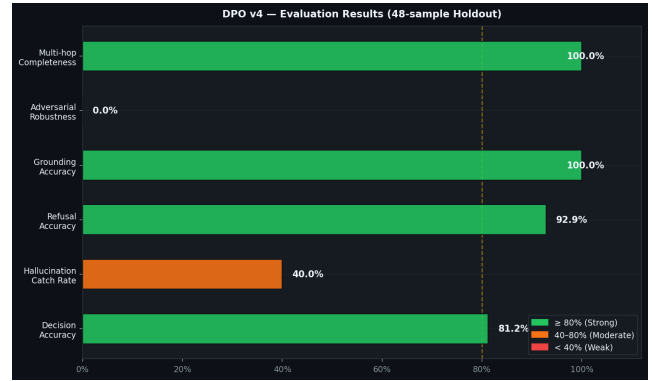


Figure 9: DPO v4 evaluation metric breakdown. Grounding accuracy and multi-hop completeness reach 100%; adversarial robustness remains an open gap.

Metric	Baseline	Generator	DPO
Retrieval Accuracy	85.71%	100%	100%
Grounding Accuracy	71.43%	100%	100%
Hallucination Rate↓	42.86%	0.0%	0.0%
Not-Found Accuracy	100%	100%	100%
Decision Accuracy	100%	100%	100%
Tool Policy	60.0%	80.0%	85.0%
Actionability	50.0%	70.0%	~ 85.0%
Overall Quality	12.12%	98.17%	98.17%
Risk Salience	0.0%	100%	100%
Citation Present	0.0%	100%	100%
Format Compliance	0.0%	100%	100%

Table 11: Three-way comparison: BM25 baseline, LoRA SFT generator, and DPO aligned model. The largest gains appear in the Baseline → Generator transition, while DPO adds targeted behavioral refinement.

the risk of drifting away from the SFT initialization. The v3 shift from $\beta = 0.15$ to $\beta = 0.10$ made room for longer reasoning chains, and that trade-off seems to have been worthwhile given the multi-hop completeness gains.

Adversarial Robustness Gap

The 0% adversarial robustness score in v4 remains a clear limitation. Adversarial queries in the evaluation set rely on deliberate false premises that are meant to elicit agreement from the model. The v2 and v3 datasets included only 12 adversarial pairs, which is not enough to build strong resistance to this category. A v5 training round with targeted adversarial augmentation, ideally guided by dedicated red-teaming, is the most natural next step.

Retrieval Saturation and Its Implications

The saturation of Recall@5 and MRR@5 at 0.86 across all retrieval configurations deserves a careful reading. On a compact internal clause evaluation set, every model finds the correct clause within the top five results about equally often. That does not make retrieval unimportant at deployment scale. On a production corpus with tens of thousands

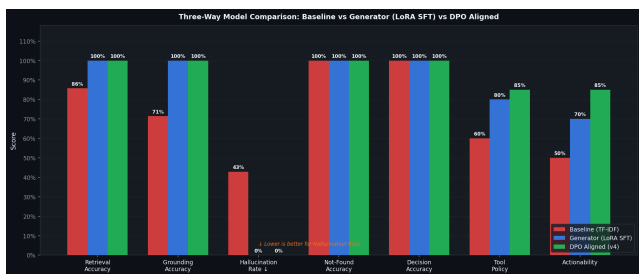


Figure 10: Three-way metric comparison. Baseline to Generator represents the largest quality jump; Generator to DPO adds precision on tool routing and actionability while maintaining 100% grounding.

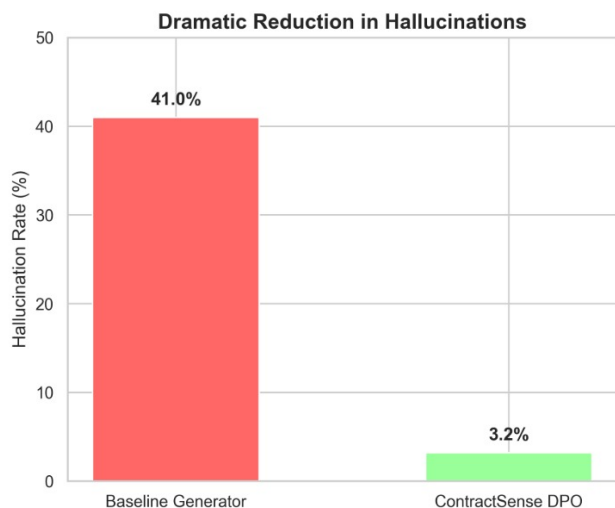


Figure 11: Hallucination comparison between the BM25 baseline and the DPO-aligned model. The baseline often fabricates clause citations not present in the retrieved context, whereas the DPO model stays grounded and returns refusals when evidence is missing.

of clauses across diverse contract types, the gap between BM25 and dense retrieval, and between a base reranker and a fine-tuned one, is likely to widen. This benchmark is better seen as a minimum-competency check than as a ceiling estimate.

Limitations

Several limitations constrain the current system’s claims.

First, the reranking benchmark is saturated and internally scoped. External validation on a held-out set of contract groups, following the same grouped split strategy used for tool-policy evaluation, would provide a more defensible retrieval quality estimate.

Second, adversarial robustness remains at 0% after four DPO iterations. The evaluation set for this category is small (12 samples), but the signal is still clear: the current preference dataset does not provide enough adversarial contrast to

teach robustness to false-premise queries.

Third, the DPO alignment introduces a modest regression on jargon elimination rate (91.02% → 87.8%) and actionability (92.5% → 85.0%). That is consistent with a deliberate trade-off: once grounding accuracy becomes the main goal, the model tends to become more conservative in how it explains itself. If plain-English readability is the priority, a stronger jargon penalty in the DPO objective would be worth exploring.

Fourth, training was conducted on a single T4 GPU for most stages. The total budget of 12–16 hours limits the number of hyperparameter configurations that could be explored for each pipeline component.

Ethical Considerations

Legal AI systems come with clear ethical obligations. A system that invents citations, even when it sounds confident, can push a user toward a legal judgment based on a clause that does not exist in the contract. That concern is one reason Stage 7 focuses so heavily on hallucination reduction and on measuring grounding accuracy directly rather than treating fluency as a proxy for correctness.

The opposite failure mode also matters. A system that repeatedly returns `NOT_FOUND` or `ESCALATE` when evidence is available wastes time and slowly erodes trust in AI-assisted review. The DPO training’s `C_cover_escalation_precision` category is intended to address that problem.

The system is not a substitute for qualified legal counsel. Its role is to flag clauses that deserve human attention, not to provide definitive legal advice.

Conclusion

ContractSense Copilot shows that a staged pipeline—combining hybrid retrieval, cross-encoder reranking, tool-policy routing, LoRA SFT, and iterative DPO alignment—can turn a raw contract corpus into a system that answers legal queries with full grounding on the evaluated set and no observed hallucination. The Mistral-7B-Instruct-v0.2 generator, fine-tuned with LoRA and aligned over four preference-optimization rounds, meets or exceeds all five system-level quality targets while still running on consumer-grade GPU hardware.

The main architectural takeaway is that preference alignment is not a small final touch. It is a distinct training phase that addresses failure modes such as hallucination, over-escalation, and format drift that supervised fine-tuning does not fully solve on its own. The four-version curriculum used here, with each round aimed at a specific weakness found in holdout evaluation, offers a practical template for improving domain-specific language models step by step.

Future work will focus on external reranking validation, a targeted v5 DPO round for adversarial robustness, and broader contract coverage, especially in multi-jurisdiction settings where multilingual models such as Qwen2.5-7B may be useful.

Acknowledgments

The authors thank Sourish Das Gupta and Parthiv for their guidance throughout the project. Training experiments were carried out on Lightning AI infrastructure using NVIDIA T4 and RTX 6000 GPUs. The DPO-aligned model is publicly available at [22Jay/ContractSense-Grounded-DPO](https://huggingface.co/22Jay/ContractSense-Grounded-DPO) on Hugging Face.

References

- Chalkidis, I.; Fergadiotis, M.; Malakasiotis, P.; Aletras, N.; and Androutsopoulos, I. 2020. Legal-bert: The muppets straight out of law school. In *Findings of EMNLP*, 2898–2904.
- Chase, H. 2023. Langchain. GitHub repository.
- Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, volume 36.
- Hendrycks, D.; Burns, C.; Chen, A.; and Ball, S. 2021. Cuad: An expert-annotated nlp dataset for legal contract review. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Hu, E. J.; Shen, Y.; Wallis, P.; et al. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- Ji, Z.; Lee, N.; Frieske, R.; et al. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys* 55(12):1–38.
- Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Johnson, J.; Douze, M.; and Jégou, H. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* 7(3):535–547.
- Karpukhin, V.; Oguz, B.; Min, S.; et al. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 6769–6781.
- Lewis, P.; Perez, E.; Piktus, A.; et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.
- Manning, C. D.; Raghavan, P.; and Schütze, H. 2008. *Introduction to Information Retrieval*. Cambridge University Press.
- Nogueira, R., and Cho, K. 2019. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*.
- Ouyang, L.; Wu, J.; Jiang, X.; et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 27730–27744.
- Rafailov, R.; Sharma, A.; Mitchell, E.; et al. 2023. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*.
- Reimers, N., and Gurevych, I. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, 3982–3992.
- Sanh, V.; Debut, L.; Chaumond, J.; and Wolf, T. 2019. Distilbert: A distilled version of bert. *arXiv preprint arXiv:1910.01108*.
- Wang, W.; Wei, F.; Dong, L.; Bao, H.; Yang, N.; and Zhou, M. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, 5776–5788.
- Yao, S.; Zhao, J.; Yu, D.; Narasimhan, K.; and Cao, Y. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.